

交换排序

冒泡排序

问几个问题？

冒泡排序乍看最为简单，但请你问自己下面几个问题：

- 1、冒泡排序如何判断数组是否有序了呢？
- 2、冒泡排序数组 `[3,1,2,4,5,6,7,8,9]` 是否有优化方式呢？
- 3、冒泡排序最好的时间复杂度，最坏的时间复杂度，还有空间复杂度清楚吗？
- 4、如何用递归的形式实现冒泡排序？
- 5、如何使用两个栈来实现冒泡排序？
- 6、对单链表又该如何进行冒泡排序？
- 7、最后一个简单的，如何使用冒泡排序对字符串数组进行排序？

算法思想

冒泡排序是最简单的排序算法了。冒泡排序通过不断地比较两个相邻元素，将较大的元素交换到右边（升序），从而实现排序。那我们直接看例子。

5	1	4	2	8	4
---	---	---	---	---	---

我们对数组 `[5, 1, 4, 2, 8, 4]`，采用冒泡排序进行排序，注意这里的两个 **4** 的颜色是不同的，主要是为了区分两个不同的 **4**，进而解释冒泡排序算法的稳定性问题。

第一轮冒泡排序：第一步：比较 5 和 1， $5 > 1$ ，则交换 5 和 1 的位置：

1	5	4	2	8	4
---	---	---	---	---	---

第二步，比较 5 和 4， $5 > 4$ ，交换 5 和 4 的位置：

1	4	5	2	8	4
---	---	---	---	---	---

第三步：比较 5 和 2， $5 > 2$ ，交换 5 和 2 的位置：

1	4	2	5	8	4
---	---	---	---	---	---

第四步：比较 5 和 8， $5 < 8$ ，不交换

1	4	2	5	8	4
---	---	---	---	---	---

第五步：比较 8 和 4， $8 > 4$ ，交换 8 和 4：

1	4	2	5	4	8
---	---	---	---	---	---

此刻我们获得数组当中最大的元素 **8**，使用橘黄色进行标记：

第一轮冒泡结束，最大的元素8到了最后，然后对于前面5个元素，进行第二轮冒泡

最终结果：

1	2	4	4	5	8
---	---	---	---	---	---

事实上第二阶段结束，整个数组已经有序了，但是对于冒泡排序而言并不知道，她还需要通过第三阶段的比较操作进行判断。

对于冒泡排序算法而言，她是通过判断整个第三阶段的比较过程中是否发生了交换来确定数组是否有序的，显然上面的过程中没有交换操作，冒泡排序也就知道了数组有序，整个算法执行结束。

关于冒泡的优化

基本的冒泡排序的实现方式，就是两个for循环，持续比较和交换。这种实现方式有一个明显的弊端，就是不论数组是否有序，两层 for 循环都要执行一遍，而我们是希望数组有序的时候，仅进行一轮判断，或者一轮都不进行（当然不判断，排序算法是不能知道数组是否有序的）。

一次优化

这里我们增加了一个标识数组是否有序，当冒泡排序过程中没有交换操作时，`swapped = false`，也意味着数组有序；否则数组无序继续进行冒泡排序。不要小看这个变量奥，因为这个变量，当数组有序的时候，冒泡排序的时间复杂度将降至 (因为其只需要执行一遍内层的 for 循环就可以结束冒泡排序)，没有这个变量，数组有序也需要的时间复杂度。

二次优化

一次优化是为了避免数组有序的情况下，继续进行判断操作的。那么二次优化又为了什么呢？

我们看下面的例子。

4	2	1	5	6	8
---	---	---	---	---	---

经过一次冒泡后，我们会注意到一个问题，但是我们注意到，数组数组中的 `[5,6,8]` 本身已经有序，而对于有序的部分进行比较是没有意义的，相当于在白白浪费资源，有没有什么办法减少这样的比较次数呢？

换句话说，是否能够确定出已经有序部分和无序部分的边界呢？

答案当然是肯定的，这个边界就是第一趟冒泡排序的过程中最后一次发生交换的位置 `j`：也就是 1 和 4 发生交换之后，4 和 5 没有发生交换，此时 1 之后的元素为有序。

第一步：4 和 2 比较， $4 > 2$ ，交换 4 和 2，将 `LastSwappedIndex = 0`；

第二步：4 和 1 比较， $4 > 1$ ，交换 4 和 1，`LastSwappedIndex = 1`；

第三步：比较 4 和 5， $4 < 5$ ，不交换，`lastSwappedIndex` 也不更新；

第四步：比较 5 和 6，不交换，`lastSwappedIndex` 也不更新；

第五步：比较 6 和 8，不交换，`lastSwappedIndex` 也不更新；

第一趟冒泡排序结束了，这里似乎看不出和之前有什么区别，但是来看第二趟冒泡排序就不一样了，此时 `j` 的取值将从 `j = 0` 到 `j = lastSwappedIndex`，第一步：比较 2 和 1， $2 > 1$ ，交换，`lastSwappedIndex = 0`，并且第二趟冒泡也就结束了，也就说我们节省了从 2 到 6 的比较操作；

最后再来一趟冒泡排序，发现没有任何交换，所以冒泡排序结束。

相比于一次优化的实现方式，二次优化的实现方式进一步减少了不必要的执行次数，两种优化后的实现方式需要冒泡排序的趟数是一样的，本质上没有什么区别。所以即使对于一个有序的数组，两种方式的时间复杂度都是 $O(n)$ 。

复杂度分析

时间复杂度

最坏情况下（数组逆序）：

当 `i == 0` 的时候，`j` 的取值范围是从 0 到 `n - 1`，内循环的执行判断和交换操作 `n - 1` 次；当 `i == 1` 的时候，`j` 的取值范围是从 0 到 `n - 2`，内循环的执行判断和交换操作 `n - 2` 次；以此类推.....当 `i` 取最大值 `n - 2` 时，`j` 的取值为 1，内循环的执行判断操作 1 次；所以，整体内循环的判断语句执行次数就是： $1 + 2 + 3 + \dots + (n - 2) + (n - 1)$ 。则最坏情况下的时间复杂度为 $O(n^2)$ 量级。

最好情况下（数组有序）：

当 `i == 0` 的时候，`swapped = false`，`j` 的取值范围是从 0 到 `n - 1`，内循环的执行判断操作 `n - 1` 次，但是没有发生交换操作，冒泡排序算法直到数组已经有序，所以执行结束。则最好情况下的时间复杂度为 $O(n)$ 。

空间复杂度

冒泡排序没有使用任何额外的空间，空间复杂度为 $O(1)$ ，是典型的 **原地排序** 算法（In-place Sorting Algorithm）。

稳定性分析

我们可以发现，两个 **4** 的相对位置没有发生变化，也就是说冒泡排序是稳定的。但这仅相当于实验验证，而在理论上冒泡排序为什么是稳定的呢？

本质原因在于冒泡排序比较和交换的是两个相邻元素，对于键值相同的关键字是不交换位置的，所以排序前后键值相同的关键字的相对位置才保持不变的。

如何用两个栈实现冒泡

对于这个问题本身，你可能会觉得没有任何意义，但是当你去努力实现的时候，就会发现，你对于栈和冒泡排序的理解有了新的见解。

问题本身不难理解，就是利用两个栈，然后每一次选择出数组中最大的元素，并存入数组对应的位置。但是当你自己去实现时，还是会发现好多问题，比如如何互换着使用两个栈？如何对栈中相邻的两个元素比较大小，并交换位置？

记得自己尝试着实现一下，一定对你的学习、面试或考试有帮助。

下面是参考的思路：

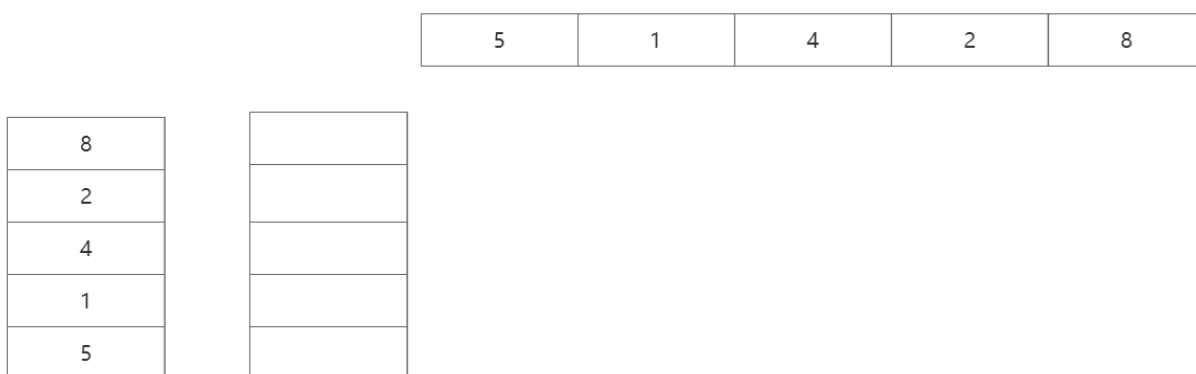
给定两个栈 `s1` 和 `s2`，以及一个长度为 `n` 的数组 `arr`：

- 1、将数组 `arr` 中的所有元素压入栈 `s1` 当中；
- 2、执行 for 循环 `n` 次（每一次选择一个最大的元素）：
 1. 情况一：`s1` 不为空，`s2` 为空，则尝试将栈 `s1` 当中的所有元素压入栈 `s2`，并保证 `s2` 的栈顶元素为最大值；当 `s1` 为空时，`s2` 中的栈顶元素即为栈中元素的最大值，插入数组相应位置。
 2. 情况二：`s2` 不为空，`s1` 为空，则尝试将栈 `s2` 当中的所有元素压入栈 `s1`，并保证 `s1` 的栈顶元素为最大值；当 `s2` 为空时，`s1` 中的栈顶元素即为栈中元素的最大值，插入数组相应位置。

详细解析

初始时两个栈 `s1` 和 `s2` 都为空栈，数组 `arr[] = [5,1,4,2,8]`。

第一步：将数组 `arr` 中的所有元素都压入栈 `s1` 当中：



第二步：栈 `s2` 为空，直接将 `s1` 的栈顶元素 **8** 压入栈 `s2`

5	1	4	2	8
---	---	---	---	---

2	
4	
1	
5	8

第三步：栈 `s1` 不为空，尝试将 `s1` 的栈顶元素 `2` 压入栈 `s2`，但是此时 `s2` 的栈顶元素 `8` $>$ `2`，所以利用一个临时变量 `tmp` 交换两个元素在栈中的位置，先将 `s2` 的栈顶 `8` 保存到 `tmp` 并弹出，然后压入元素 `2`，最后再将 `8` 重新入栈。(其实就是交换操作)

5	1	4	2	8
---	---	---	---	---

4	
1	8
5	2

temp

第四步：栈 `s1` 不为空，同第三步一样将 `s1` 的栈顶元素压入栈 `s2` 当中：

5	1	4	2	8
---	---	---	---	---

1	8
5	4

temp

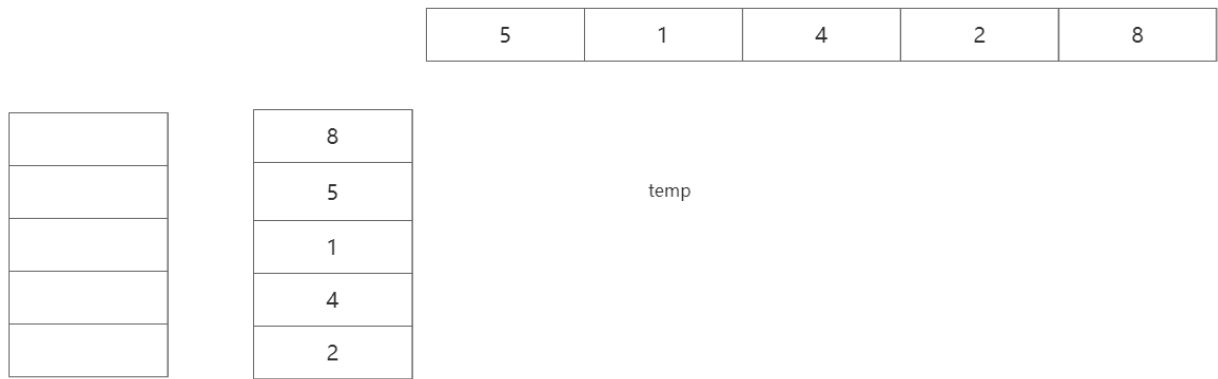
第五步：栈 `s1` 不为空，同上将 `s1` 的栈顶元素 `1` 压入栈 `s2` 当中：

5	1	4	2	8
---	---	---	---	---

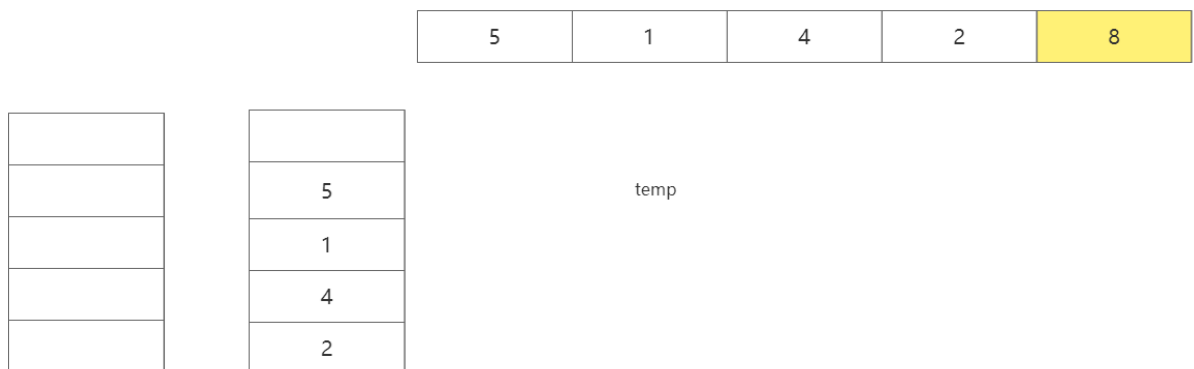
	8
	1
	4
5	2

temp

第六步：栈 `s1` 不为空，同上将 `s1` 的栈顶元素 **5** 压入栈 `s2` 当中：



第六步：栈 `s1` 为 `空`，弹出 `s2` 的栈顶元素，并将其放到数组 `arr[n - i - 1]` 的位置：



之后的过程和前面讲的类似，将栈 `s2` 中的元素压入栈 `s1` 当中，并找到次大元素 **5**，以此类推，实现对数组的冒泡排序。

快速排序

算法思想

快速排序首先选一个基准（你也可以认为是要放到排序后数组正确位置的元素）`pivot`，然后将数组按照选取的基准 `pivot` 进行划分。而选取 `pivot` 的方式又有很多种，所以快速排序具有很多版本。

总是选择第一个元素作为基准 `pivot`；

总是选择最后一个元素作为基准；（以这个举例）

随机的选择一个元素作为基准；

选择最中间的元素作为基准；

快速排序的关键是划分 `partition()`。每一趟划分，我们就可以将作为 `pivot` 的值 `x` 放到排序数组的正确位置，并且将所有比 `x` 小的放到 `x` 的左边，所有比 `x` 大的元素放到 `x` 的右边。而且划分操作的时间复杂度是线性的奥，即 $O(n)$ 量级！

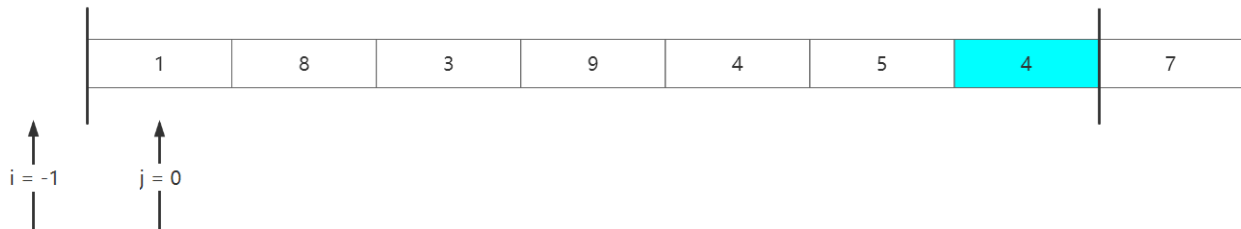
正所谓一图胜千文，我们看图学算法吧！

为了讲解快速排序并验证其稳定性，我们用下面的数组进行说明（其中两个4分别用不同的颜色标注）：

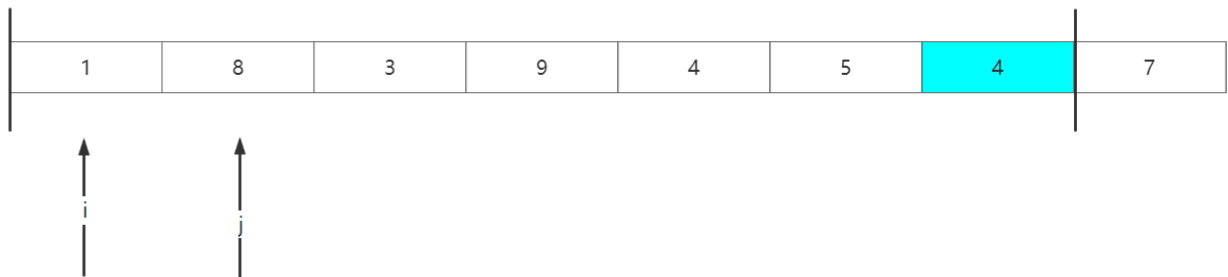
1	8	3	9	4	5	4	7
---	---	---	---	---	---	---	---

首先选择数组当中的最后一个位置元素 **7** 作为 pivot：然后就是执行第一趟快速排序啦！！

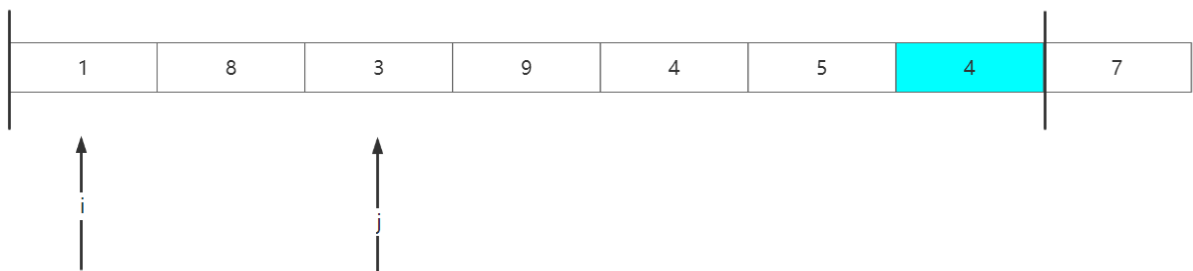
第一步：设置一个指针 **$i = -1$** （初始化为 -1，用于找到 pivot 的正确位置），设置一个遍历数组的指针 **$j = 0$** ，用于遍历从 0 到 pivot 的前一个元素 **4**（即两条竖线之间的元素，从而将 **7** 放到排序后数组的正确位置）：



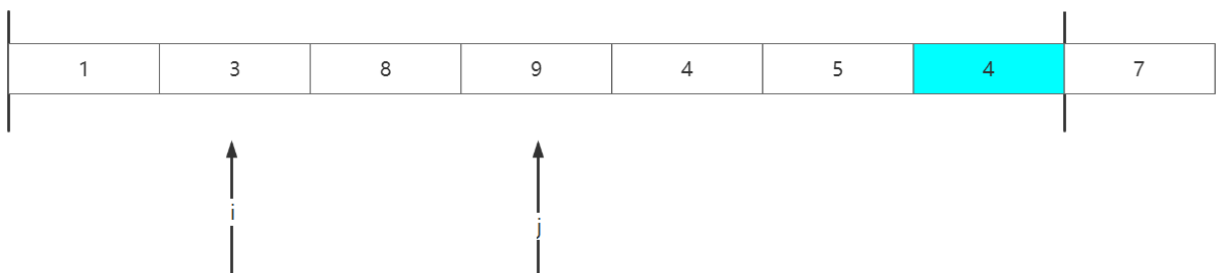
第二步：比较 **j** 当前指向的元素 **1** 和 **7**， **$1 \leq 7$** ；指针 **$i++$** ，即 **$i = 0$** ，交换 **$arr[i]$** 和 **$arr[j]$** ，即交换 **$arr[0]$** 和 **$arr[0]$** （数组本身并无变化）；便利然后指针 **j** 向右移动：



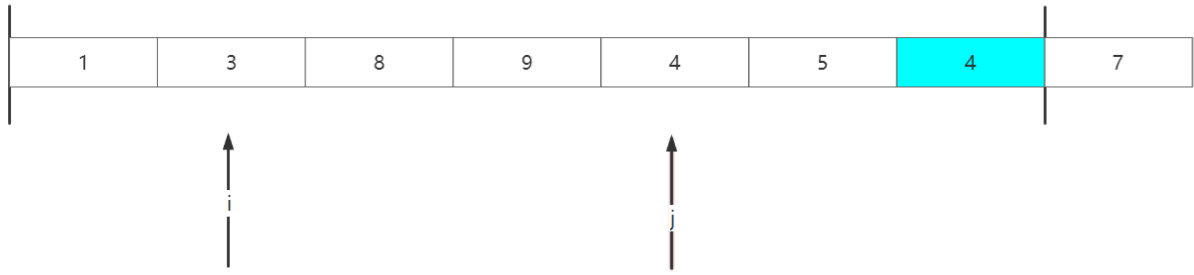
第三步：比较当前 **j** 指向的元素 **8** 和 **7**（pivot）， **$8 > 7$** ；什么都不做；然后指针 **j** 向右移动：



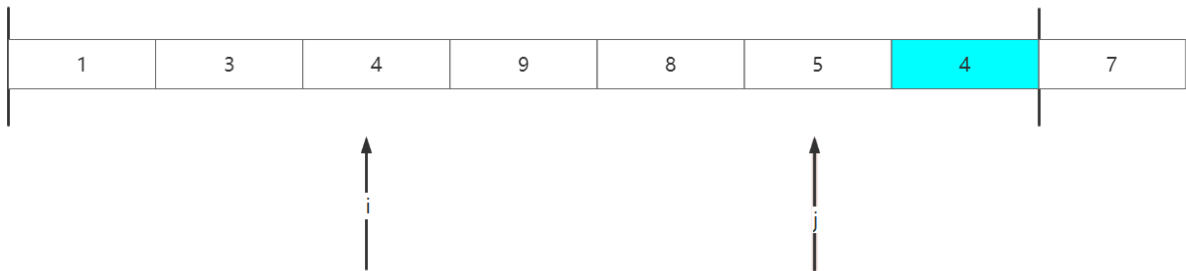
第四步：比较当前 **j** 指向的元素 **3** 和 **7**， **$3 \leq 7$** ；指针 **$i++$** ，即 **$i = 1$** ，交换 **$arr[i]$** 和 **$arr[j]$** ，即交换 **$arr[1] = 8$** 和 **$arr[2] = 3$** ；然后指针 **j** 向右移动：



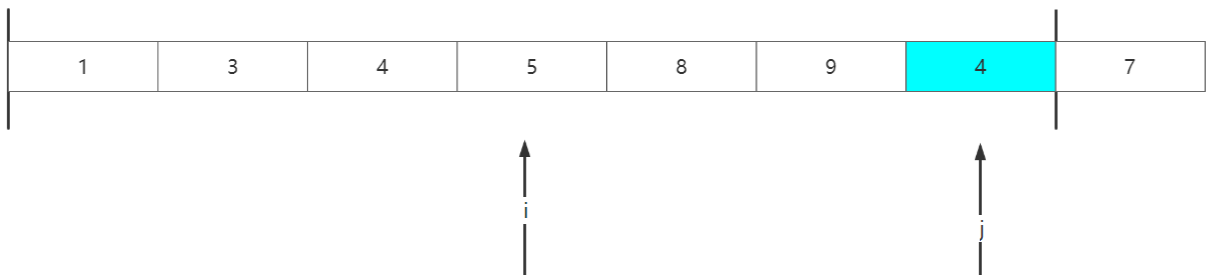
第五步：比较当前 **j** 指向的元素 **9** 和 **7**， **$9 > 7$** ；什么都不做；然后指针 **j** 向右移动：



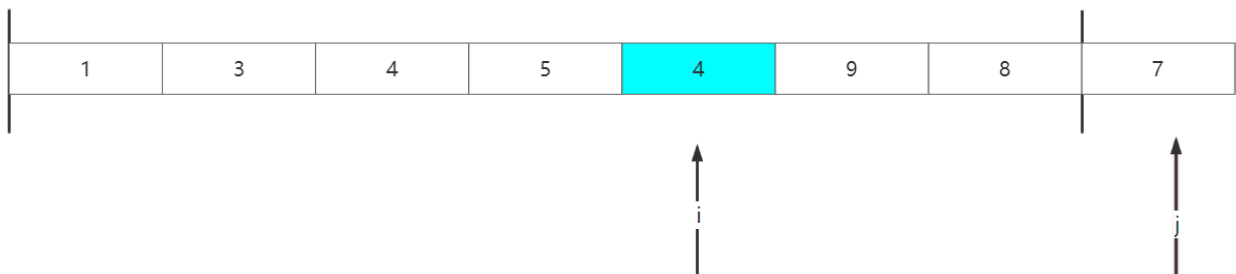
第六步：比较当前 **j** 指向的元素 4， $4 \leq 7$ ；指针 **i++**，即 **i = 2**，交换 **arr[i]** 和 **arr[j]**，即交换 **arr[2] = 8** 和 **arr[4] = 4**；然后指针 **j** 向右移动：



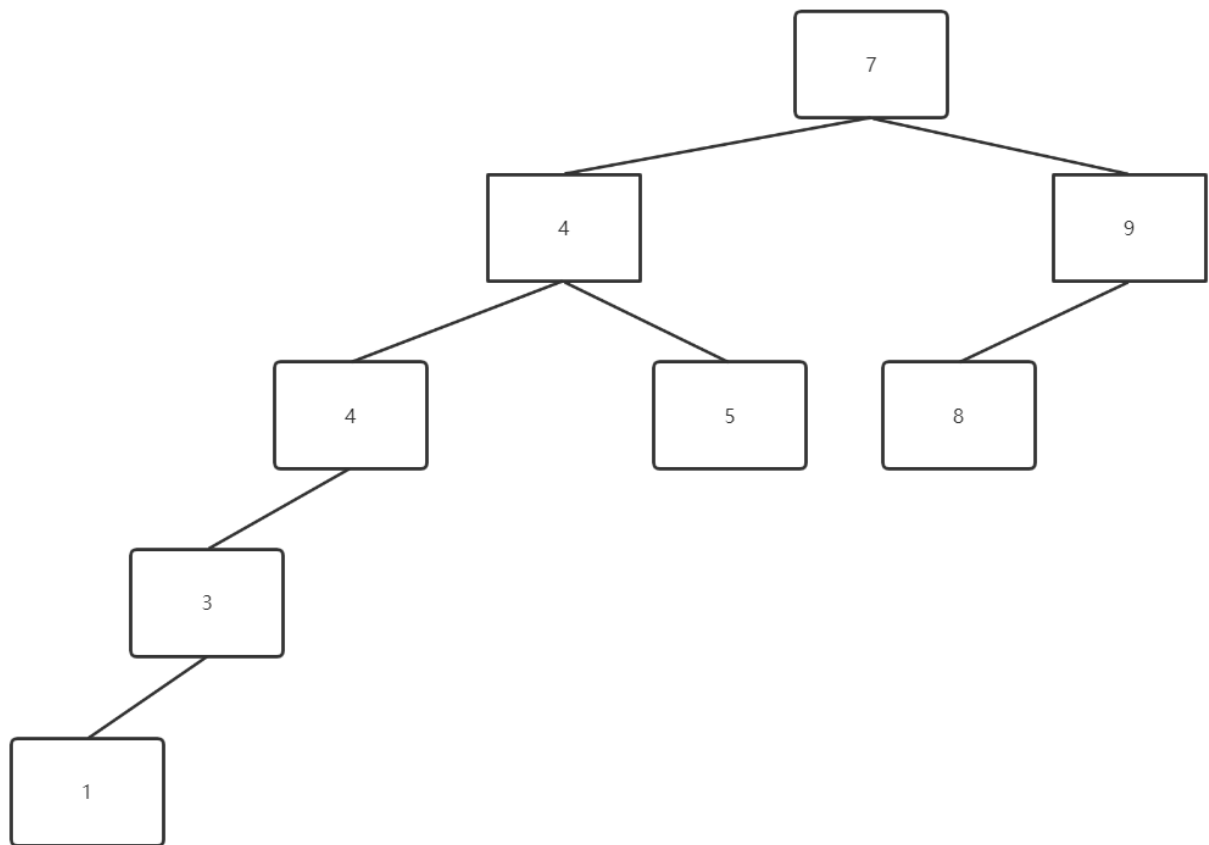
第七步：比较当前 **j** 指向的元素 5 和 7， $5 \leq 7$ ；指针 **i++**，即 **i = 3**，交换 **arr[i]** 和 **arr[j]**，即交换 **arr[3] = 9** 和 **arr[5] = 5**；然后指针 **j** 向右移动：



第八步：比较当前 **j** 指向的元素 4 和 7， $4 \leq 7$ ；指针 **i++**，即 **i = 4**，交换 **arr[i]** 和 **arr[j]**，即交换 **arr[4] = 8** 和 **arr[6] = 4**；然后指针 **j** 向右移动：

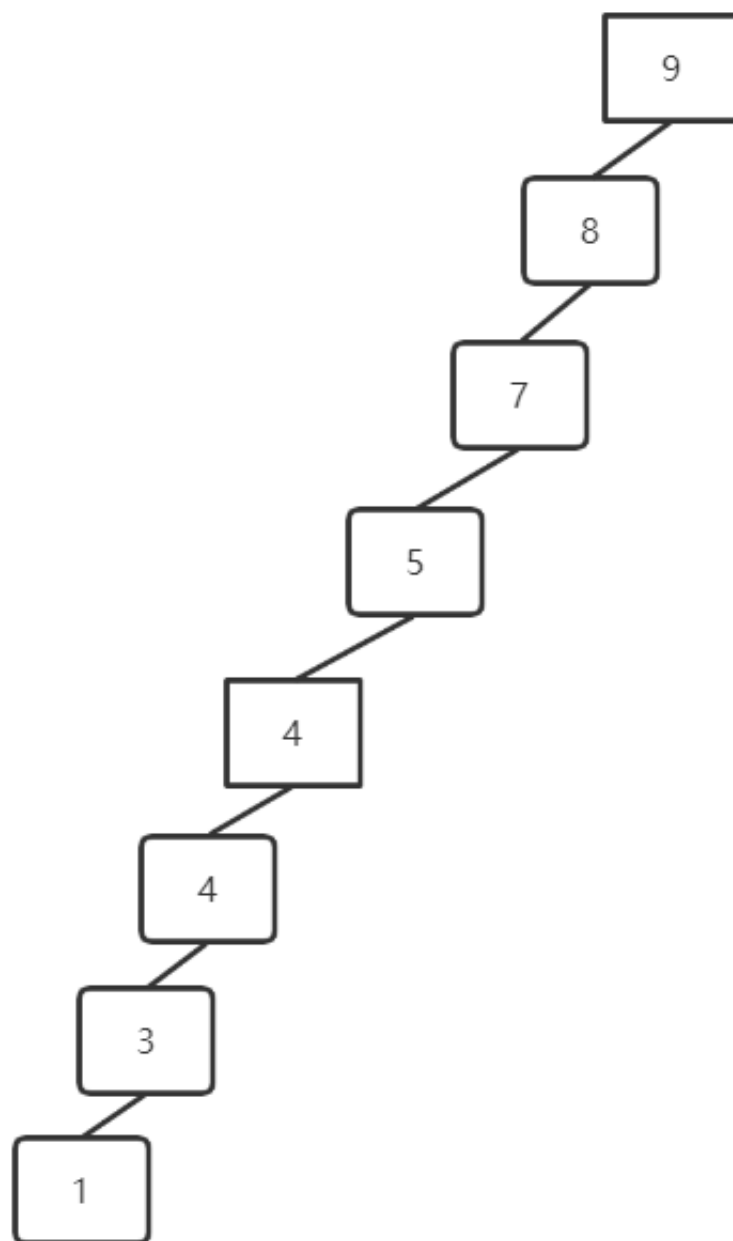


第九步：此时遍历结束，交换 **arr[i+1]** 和 **arr[high] = pivot**，即交换 9 和 7



的确如此，只不过对于这个数组而言选择最末尾的元素作为 pivot 得到的树的高度并不是我们期望的 $\log n = \log 8 = 3$ ，而是 **4**。

说到这里，我们顺带说一下快速排序的缺点，对于一个有序数组 `[1,3,4,4,5,7,8,9]` 而言，如果每次选择最后一个元素作为 `pivot`，就会得到下面一幅图：



而这时树的高度变成了 n ，也就意味着快速排序退化成了一颗单链，这不是我们希望看到的。但是我们每一次选择最中间的元素作为 `pivot`，又会怎么样呢？

如果将数组 `[1, 8, 3, 9, 4, 5, 4, 7]` 重新调整顺序，使得快速排序的分治过程如上图所示？看图就能写出来了，当然答案可能有很多个，最简单的一个就是 `[1, 4, 3, 5, 8, 9, 7, 4]`。

复杂度分析

快速排序的时间通常表示为： $T(n) = T(k) + T(n - k + 1) + n$

其中 $T(k)$ 和 $T(n - k + 1)$ 分别表示递归调用，而最后一项 n 表示 将最后一个元素作为`pivot`进行划分 的处理过程， k 表示比 `pivot` 小的元素的数目。

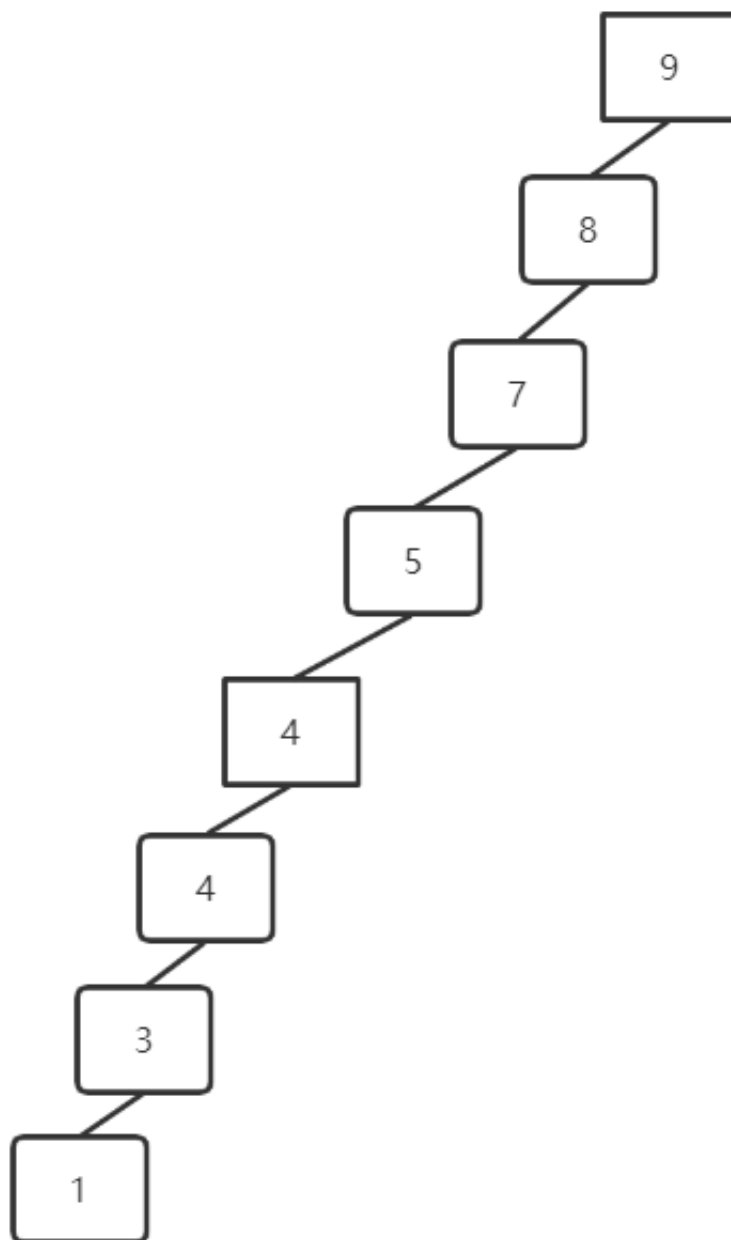
而快速排序的时间复杂度取决于输入的数组和划分策略，所以需要从三个方面分析：

最坏情况

最后情况就是我们每一次选择最大的元素或者最小的元素作为 `pivot`。以我们上面讲快速排序选择做末尾的元素作为 `pivot`，最坏情况就是输入的待排序数组为有序数组（以升序为例），此时 $k = n - 1$ ，那么： $T(n) = T(n - 1) + T(0) + n$ ，即 $T(n) = T(n-1) + n$ ；

所以最坏情况下的时间复杂度为 $O(n^2)$ 量级。

不理解推导没关系，看栗子，设对有序数组 `[1,3,4,4,5,7,8,9]` 进行快速排序，每次选择最末尾的元素作为 `pivot`，那么就会得到下图所示的情况：



也就是说需要选择 n 个 `pivot`，并且以每一个 `pivot` 进行划分需要 $O(n)$ 的时间，那么总的时间就是 $O(n^2)$ 量级。

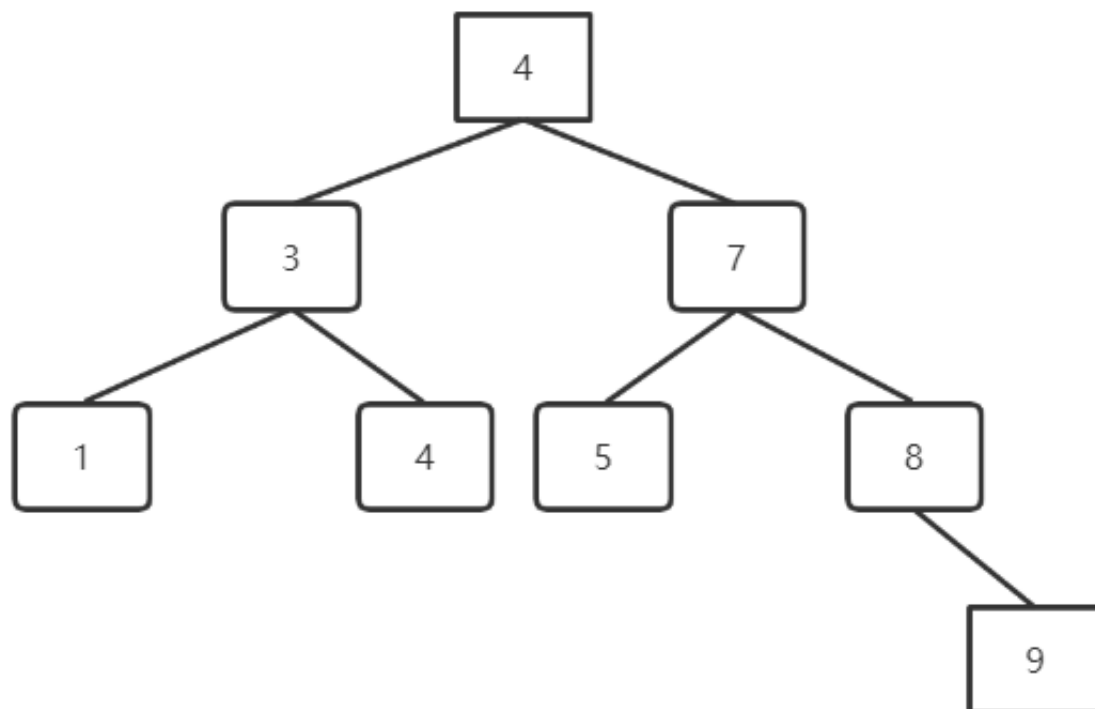
最好情况

当划分过程中每一次都能选择最中间的元素作为基准 `pivot`，那么快速排序的时间复杂度就等于： $T(n) = T(n/2) + T(n/2) + n$

其中 $T(n)$ 表示快速排序的时间复杂度， $T(n/2)$ 表示划分出的两个子数组排好序所用的时间， n 表示将最后一个元素作为`pivot`进行划分函数的执行时间。

根据主定理（Master Theorem），快速排序最好情况下的时间复杂度为 $O(n \log n)$ 。

当然我们也可以换一个角度来算，比如对数组 `[1, 8, 3, 9, 4, 5, 4, 7]` 而言，我们希望得到的是下面一幅图：



这个树的高度就是 $O(\log n)$ ，也就是选择 `pivot` 需要 $O(\log n)$ 次，而根据每一个 `pivot` 我们需要 $O(n)$ 的时间执行划分函数，所以总的时间复杂度为 $O(n \log n)$ 量级。

平均情况

对于平均时间复杂度分析而言，我们需要考虑数组的所有可能的排列，并计算出对每一个排列所需要的时间，然后求平均，但是实在太复杂了。我们可以考虑一个一般的假设，比如对于一个数组而言， $n/10$ 的元素每次比选择的 `pivot` 小，而 $9n/10$ 的元素比 `pivot` 大，那么快速排序的时间复杂度为： $T(n) = T(n/10) + T(9n/10) + n$ 。

根据主定理，快速排序的时间复杂度依旧是 $O(n \log n)$ ，也就意味着只要只要每一次不是选择最大或者最小的元素作为 `pivot`，时间复杂度都在 $O(n \log n)$ 量级。

快速排序的平均时间复杂度为 $O(n \log n)$ 量级。

空间复杂度分析

快速排序的实现中，我们仅使用了一个临时变量用于交换操作，也就是其空间复杂度为 $O(1)$ ，所以快速排序也是一个原地排序算法。

稳定性分析

快速排序的划分阶段会进行交换操作，而这种交换操作会破坏原始数组中元素之间的相对位置，也就意味着，快速排序是一个不稳定的排序算法。

当然所有的排序算法都可以变得稳定，有机会我们再谈稳定的快速排序如何实现。

快速排序的稳定性从哪里来？

快速排序是不稳定的。这个论述在标准的教科书上已经提了很多次，也没有什么疑问。

但是，可否通过改进的方法将其变成一个稳定的排序算法呢？

快速排序的核心过程是 Partition 函数，这个函数的作用是选择一个基准（pivot），使比之小（或等于）的数都在结果列表中处于其左边，比之大的数都在结果列表中处于其右边。我们发现，快速排序不稳定性的根源就来自于此。比如这个例子：

【1, 4, 2-1, 3, 6, 2-2, 5, 2-3】

列表中出现了三个2，那么我们如果选择中间的2（绿色）作为基准，则第一次Partition结束的列表成为

【1, 2-1, 2-3, 2-2, 4, 3, 6, 5】

或者

【1, 2-3, 2-1, 2-2, 4, 3, 6, 5】

可见，黄色和蓝色的2都被分在了绿色2的左边或者右边，明显三个2的相对顺序变化了。

结论：如果一个数字在待排序的列表中出现三次或以上，而这个数字在列表中出现（非首次和末次）一次被选为基准（pivot），则结果肯定是不稳定的。

因此，通过更改比较时 $\geq$ 符号为 $>$ 符号是没有意义的，因为不稳定的根源在于基准的选取。

如果要得到稳定的快速排序，必须在选取基准时避免这种情况，而且在交换元素时要小心不要打乱相同元素的相对顺序。

一种可选的做法是先记录相同元素的相对位置，再进行排序。这需要额外的空间开销。